

Name: _____

Write your name and computing ID above. Write your computing ID at the top of each page in case pages get separated. Sign the honor pledge below.

Generally, we will not answer questions about the exam during the exam time. If you think a question is unclear and requires additional information to answer, please explain how in your answer. For multiple choice questions, write a * next to the relevant option(s) along with your explanation.

On my honor as a student I have neither given nor received aid on this exam.

1. (12 points) Suppose we have a C project which builds two applications:

- **foo** is built from the source files `foo.c` and `util.c`, and for **foo**, `util.c` needs to be compiled with the compiler flag `-DFOO`
- **bar** is built from the source files `bar.c` and `util.c`, and for **bar**, `util.c` needs to be compiled with the compiler flag `-DNOTFOO`

`foo.c`, `bar.c`, and `util.c` all include a header file `util.h`.

Fill in the blanks to complete the Makefile for building this C project as specified above. **You may not need all lines.**

```
foo: foo.o util-foo.o
    clang -o foo foo.o util-foo.o
```

```
bar: bar.o util-bar.o
    clang -o bar bar.o util-bar.o
```

```
util-foo.o: util.c util.h
    clang -DFOO -c util.c -o util-foo.o
```

```
util-bar.o: util.c util.h
    clang -DNOTFOO -c util.c -o util-bar.o
```

```
-----: -----
```

```
-----
```

```
foo.o: foo.c util.h
    clang -c foo.c -o foo.o
```

```
bar.o: bar.c util.h
    clang -c bar.c -o bar.o
```

2. (8 points) Consider the following C program:

```
int x = 0;

int main() {
    int y = 0;
    pid_t p = fork();
    if (p == 0) {
        x += 3;
        y += 3;
        exit(0);
    } else {
        pid_t q = fork();
        if (q == 0) {
            x += 2;
            y += 2;
        } else {
            x += 10;
            y += 10;
        }
    }
    printf("x:%d y:%d\n", x, y);
}
```

Which of the following are lines this program could print? **Select all that apply.**

- ☐ x:0 y:0
- ☐ x:0 y:10
- ☒ x:2 y:2
- ☐ x:3 y:3
- ☒ x:10 y:10
- ☐ x:15 y:15
- ☐ x:15 y:10

3. (12 points) We are developing a new shell (csosh) and want to add a new operator for the command line. The operator is “@” and redirects both stdout and stderr a file.

The user runs `./myprogram @output.txt` in the csosh shell. Assume there is code not shown that parses the input. Fill in the blanks to make the @ operator work and for “Command finished” to print after myprogram runs. **You may not need all lines.**

```
    ello

char *prog /* = "./myprogram" */;
char *args[] /* = {"./myprogram", NULL} */;
char *at_filename /* = "output.txt" */;

pid_t p = fork();
if (p > 0) {
    int status=0;
    waitpid(p, &status, 0);

    -----
} else {
    int fd = open(at_filename, O_WRONLY);
    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);
    close(fd);
    execv(prog, args);
}
printf("Command finished.\n");
```

4. Consider a system with 12-bit page offsets. Suppose on that system, the following is part of a process’s page table:

virtual page number	valid	writable	physical page number
...	...	...	...
0x200	1	0	0xAB9
0x201	0	0	---
0x202	0	0	---
0x203	1	0	0xC08
0x204	1	0	0xAB5
...	...	...	...
0x7F0	0	0	---
0x7F1	1	1	0x99D
0x7F2	1	1	0x99A
...	...	...	...

and the operating system has assigned:

- addresses 0x200000—0x202FFF: loaded from `foo.exe` on demand, read-only
- addresses 0x203000—0x204FFF: loaded from `foo.exe` on demand, copy-on-write copy of the file
- addresses 0x7F0000—0x7F2FFF: stack space, allocated on demand

Assume the operating system implements the “allocate/load on demand” and “copy-on-write” strategies we discussed in lecture, allocating or loading or copying only one page at a time.

- (a) (4 points) Which of the following is true about what happens if the program tries to write to address 0x204880? *-1 per wrong* **Select all that apply.**

☒ an hardware exception will occur
 ☐ the program will crash
 ☐ new data will be loaded from `foo.exe` for VPN 0x204
 ☒ a copy of the data in physical page 0xAB5 may be made
 ☐ the page table entry for VPN 0x200 will become writable
 ☐ the page table entry for VPN 0x202 will become valid
 ☐ the page table entry for VPN 0x203 will become valid
 ☒ the page table entry for VPN 0x204 will become writable
- (b) (6 points) Which of the following is true about what will be true after the program forks and writes to 0x7F0800 from the child process? Assume the program does not perform other memory accesses. *-1 per wrong* **Select all that apply.**

☐ the child process will have crashed
 ☒ the page table entry for VPN 0x7F0 in the child process will be valid
 ☐ the page table entry for VPN 0x7F0 in the parent process will be valid
 ☒ the page table entry for VPN 0x7F0 in the child process will be writable
 ☐ the page table entry for VPN 0x7F0 in the parent process will be writable
 ☒ the page table entry for VPN 0x7F1 in the child process will be valid
 ☒ the page table entry for VPN 0x7F1 in the parent process will be valid
 ☐ the page table entry for VPN 0x7F1 in the child process will be writable
 ☐ the page table entry for VPN 0x7F1 in the parent process will be writable
 ☐ the parent’s page table entry for VPN 0x7F1 will have a different physical page number than the child’s page table entry for 0x7F1

5. For the following questions consider a directory with the following contents:

permissions			owner	group	name
user	group	other			
rwX	—	—	cso6p	faculty	file1
rw-	r-	r-	cso6p	faculty	file2
rwX	r-X	r-X	cso6p	faculty	file3
rwX	r-X	r-X	cso6p	faculty	file4
rw-	r-	r-	cso6p	faculty	file5

- (a) (4 points) After these commands are run, what are the file permissions for **file2** and **file3**? Specify your answers using octal notation.

```
chmod 117 file2
chmod o=r file2 file3
```

file2 permissions: 114

file3 permissions: 754

- (b) (8 points) A new user named **dsa3r** needs write access to **file4**. What are two different methods for enabling that?

Solution: Add the user **dsa3r** to the **faculty** group and give the **faculty** group write access to **file4**, change the owner of **file4** to **dsa3r**, or give other the write permission for **file4**.

6. Assume a system uses 1024-byte (2^{10} -byte) pages and the following C function runs in a process on the system.

```
uint32_t data[8000];
void example_function() {
    int sum = 0;
    printf("%#:02p\n", &data[0]);
    for (int i=0; i<4096; i+=1024) {
        for (int j=0; j<6; j++) {
            sum += data[i+j];
        }
    }
}
```

`uint32_t` is a 32-bit (4-byte) integer type. When the `printf()` statement ran it printed `0x3f9708`.

- (a) (6 points) How many pages will be accessed by the accesses to `data`?

Solution: 4

- (b) (6 points) Assume this program uses one page for its code, one page for its heap, one page for its stack, and 40 pages for its static data. If a system only needed to store memory for just this execution of just this program and kept all of the program's data in physical memory, what is the smallest physical address space the system could use? Specify your answer in terms of the number of total bits in the address.

Solution: 16 bits

- (c) Assume the system running this program uses 28 bit virtual addresses and 16 bit physical addresses. Consider the memory access of `data` when `i=2048` and `j=3`.

- i. (5 points) Which virtual address will the program access? You may leave your answer as an unsimplified arithmetic expression.

Solution: `0x3fb714`

- ii. (3 points) What is the VPN of the address from (i)? You may leave your answer as an unsimplified arithmetic expression.

Solution: `0x0fed`

7. Suppose the following sequence of events happens on a single-core system.
1. initially, a program that is performing a large computation is active on the processor
 2. a user inputs a command to run a compiler
 3. the shell processes that command and starts the compiler
 4. the compiler requests to read the source code from disk
 5. the operating system does not have the source code immediately available and makes a request for it to the disk
 6. while it is waiting for the disk to read the source code, the operating system runs a program that is performing a large computation
 7. then, shortly after the disk read completes, the compiler resumes running and processes the source code that was just read
 8. the compiler writes its output file
 9. the compiler exits
 10. the shell resumes running and displays its next prompt
- (a) (4 points) When the compiler started running in step 7 above, its register values were loaded onto the processor. Which of the following most likely loaded those values?
- ☐ the processor when it restored values from a second ‘backup’ set of registers when it received a signal from the disk
 - ☐ an operating system routine that started running when the compiler made a system call asking to read from the disk
 - ☒ an operating system routine that the processor started running when the disk indicated data was available
 - ☐ an operating system routine that started running when the large computation program divided by zero
 - ☐ the processor when the operating system routine that handles disk input made a system call requesting to switch processes
 - ☐ the processor when it restored values from a second ‘backup’ set of registers after a timer expired
 - ☐ an operating system routine that started running when the disk accessed unallocated memory
 - ☐ an operating system routine that started running when the large computation program made a system call
 - ☐ an operating system routine that started running when the large computation program accessed unallocated memory
- (b) (6 points) During which of the steps above did a hardware exception triggered by an external device most likely happen? For each such step, identify the relevant external device.

Solution: step 2 (keyboard); step 7 (disk)
